

CONTROL EDGE | EPISODE 2

בתוך הבקר התעשייתי

ארכיטקטורת PLC, PAC ומחשב תעשייתי - תזמון, זיכרון ו-I/O

מהנדסי מכונות, תהליך, ייצור, מכשור ובקרה ואינטגרטורים	קהל יעד
40 עד 44 דקות	משך יעד
שיחה בין יעל - מהנדסת מכונות צעירה, ואמיר - מהנדס תהליך ובקרה ותיק	פורמט
מכונת מילוי עם שולחן אינדקס מבוקר סרוו ומערכת סחרור, המשלבת Motion, רצפים ובקרת תהליך	דוגמה מרכזית
הפרק השני בסדרת Control Edge	מיקום בסדרה

חומר לימודי. אינו מחליף תכן הנדסי, הערכת סיכונים, הוראות יצרן או את המהדורות החוזיות והמחייבות של התקנים הרלוונטיים.

בדיקת כיוון מול מסמך האב

פרק 1 הסתיים בהבטחה לפתוח את ארון הבקרה. לכן פרק זה נשאר מתחת לשכבות HMI ו-SCADA ומתמקד בבקר עצמו: חומרה, תזמון, זיכרון, I/O וזמינות. IEC 61131-3 מוצג כמסגרת התכנות בלבד; תכנון קוד, מוסכמות כתיבה ובדיקות תוכנה יעמדו במרכז פרקים מאוחרים יותר.

1. בדיקת התאמה לפני הפקה

- הקהל נשאר מהנדסי מכונות ותהליך, ולא רק מתכנני אוטומציה.
- הפרק מעמיק בבקר עצמו ואינו קופץ מוקדם מדי ל-HMI, SCADA או ענן.
- הדוגמה משלבת Motion, לוגיקה דיסקרטית ובקרת תהליך כדי לחבר בין תחומי הקהל.
- התקנים מוצגים כמסגרות מקצועיות, בלי להמציא סעיפים או לטעון לתחולה חוזית.
- הסיום מכין את פרק 3 על חיישנים ואיכות אות.

2. מטרות הפרק

- להסביר מה מבדיל בקר תעשייתי ממחשב רגיל.
- להבחין בין PLC, PAC, מחשב תעשייתי, Soft PLC, בקר DCS ובקר בטיחות בלי להסתמך על מונחים שיווקיים.
- לתאר את חומרת הבקר: ספק כוח, CPU, זיכרון, Backplane, תקשורת ו-I/O מקומי או מרוחק.

- להסביר Scan Cycle, משימות מחזוריות ומונעות אירוע, עדיפויות, Interrupts, Watchdog, Jitter וזמן תגובה מקצה לקצה.
- להבחין בין זיכרון נדיף, בלתי נדיף Retentive-1 ולהבין את משמעות ההתנעה מחדש.
- להציג את המהדורה הרביעית של IEC 61131-3 משנת 2025 ואת תפקידי ST, LD, FBD ו-SFC.
- לבנות ציקליסט מעשי לבחירת בקר הכולל ביצועים, סביבה, מחזור חיים, סייבר ותחזוקתיות.

3. חלוקת זמן ומקטעים

מטרה	מקטע	זמן
להראות שמהירות CPU ממוצעת אינה מבטיחה תגובה דטרמיניסטית.	פתיחה: שער הפסילה שאיחר	00:00-02:30
להגדיר גבולות בין PLC, PAC, IPC, Soft PLC, DCS ובקר בטיחות.	מה נחשב בקר?	02:30-05:30
למפות CPU, זיכרון, Backplane, מתח, תקשורת ו-I/O.	בתוך החומרה	05:30-10:00
להסביר Scan, Tasks, עדיפויות Interrupts-1.	מודל הביצוע	10:00-15:30
לקשור בין Cycle Time, Jitter, Latency והתגובה הפיזיקלית.	דטרמיניזם ו-Watchdog	15:30-19:30
להסביר מסלולי עדכון, דיאגנוסטיקה והתנהגות בכשל.	I/O מקומי ומרוחק	19:30-24:00
להפריד בין נתונים נדיפים, בלתי נדיפים Retentive-1.	זיכרון והתנעה מחדש	24:00-28:00
להציג שפות, POU, שינוי Online ומגבלות ניידות.	IEC 61131-3 וכלי הפיתוח	28:00-32:00
לבחור ארכיטקטורה לפי דרישות ולא לפי תווית.	PLC מול PAC מול IPC	32:00-36:00
להסביר מה מוכפל ומה נשאר נקודת כשל יחידה.	יתירות וזמינות גבוהה	36:00-39:30
להפריד בין בקרה בסיסית, FS-	גבולות בטיחות וסייבר	39:30-42:00

מטרה	מקטע	זמן
PLC ותפעול מאובטח.		
להפוך את המושגים לשיטת עבודה הנדסית.	מפת Tasks וצ'קליסט סיום	42:00-44:00

4. מפת בחירת ארכיטקטורה

אין גבול תקני חד בין PLC ל-PAC. הטבלה מיועדת לתרגם שמות משפחה לדרישות שניתן למדוד, לבדוק ולתחזק.

ארכיטקטורה	חוזקות אופייניות	שאלות לפני בחירה
PLC קומפקטי או מודולרי	לוגיקה תעשייתית עמידה ודטרמיניסטית, I/O ישיר, מחזור חיים ארוך ותחזוקה מוכרת	כמות וסוגי I/O, זמני Tasks, תנאי סביבה, תקשורת, הרחבה ותמיכה?
PAC	מונח ספקים נפוץ לפלטפורמות גדולות, עשירות בנתונים ורב-תחומיות	איזו יכולת מדידה מסתתרת מאחורי השם: זיכרון, Multi-tasking, יתירות, Motion, ספריות תהליך או רשתות?
מחשב תעשייתי עם Runtime בזמן אמת	כוח חישוב, שילוב תוכנה פתוח, ראייה ממוחשבת, אנליטיקה Multi-core-1	כיצד זמן האמת מבודד ממערכת ההפעלה? מהי אסטרטגיית התאוששות, עדכונים ואחסון?
בקר DCS	ספריות תהליך, בסיס נתונים הנדסי, אזעקות, Historian ומחזור חיים מפעלי	האם נדרשת תזמור תהליכי ברמת יחידה וזמינות גבוהה יותר מהתמחות במחזור מכונה?
Safety PLC / FS-PLC	תת-מערכת לוגית מאושרת לפונקציות בטיחות עם כלים ודיאגנוסטיקה מבוקרים	אילו פונקציות בטיחות, יעדי שלמות, עצמאות, Proof Test וראיות מחזור חיים נדרשים?

הסתייגות הנדסית

שם הבקר אינו מוכיח ביצועי זמן אמת, שלמות בטיחותית, זמינות או סייבר. יש לאמת את הפלטפורמה, הקונפיגורציה, ה-Firmware, מסלול ה-I/O והאפליקציה בתנאי Worst Case.

5. הנחיה מוכנה ל-NotebookLM

הנחיה מוכנה להדבקה

צור פרק פודקאסט בעברית בלבד, באורך 40-44 דקות, המבוסס אך ורק על חבילת המקורות שהועלתה ועל מסמך זה. השתמש בשני מגישים: יעל, מהנדסת מכונות צעירה ששואלת שאלות מעשיות של תכן ותחזוקה; ואמיר, מהנדס תהליך ובקרה ותיק שמסביר ארכיטקטורה, תזמון ומחזור חיים. השיחה צריכה להישמע כמו שני מהנדסים הסוקרים מכונה אמיתית, ולא כמו הרצאה שמחולקת לשני קולות.

השתמש במכונת המילוי עם שולחן האינדקס והסחרור כדוגמה רציפה. תן ליעל כ-45% מזמן הדיבור ולאמיר כ-55%. יעל צריכה לאתגר מונחים עמומים כמו "PLC מהיר", "PAC", "זמן אמת", "יתירות" ו-"Fail Safe". אמיר צריך לענות באמצעות גדלים הנדסיים מדידים, מצבי כשל ושיטות אימות.

פתח כל ראשי תיבות בהופעה הראשונה. הבחן בין תקנים בינלאומיים לבין מימוש ספציפי של יצרן. ציין במפורש שכל זמני ה-Task, הספים והארכיטקטורות בדוגמה הם להמחשה בלבד ואינם ערכי תכן. אל תמציא מספרי סעיפים, הסמכות, ביצועי מוצר או יכולות יצרן שאינם במקורות. אל תטען ש-IEC 61131-3 מבטיח ניידות מלאה של קוד בין יצרנים.

הפרק חייב לכסות חומרת בקר, מסלולי I/O, Tasks מחזוריים ומונעי אירוע, עדיפות משימות, Watchdog, Jitter, זמן תגובה מקצה לקצה, שמירת זיכרון, התנהגות לאחר Restart, שפות IEC 61131-3, בחירה בין PLC/PAC/IPC, יתירות, גבולות בקר בטיחות וסייבר של הבקר. סיים בציקליסט בן שבע שאלות ובמעבר ברור לפרק 3 בנושא חיישנים ושלמות אות.

6. תסריט מלא לפרק

00:00-02:30 | פתיחה - שער הפסילה שאיחר

אמיר: דמייני מכונת מילוי מהירה. מערכת ראייה מזהה מיכל עם פקק לא תקין. לבקר יש שישים מילישניות להזיז שער פסילה פנאומטי לפני שהמיכל חולף על פני המסית. בזמן ההקמה זה עובד. חודשיים אחר כך, לאחר שהוסיפו חיבור ל-Historian ושגרת דיאגנוסטיקה חדשה, מיכל אחד מתוך כמה אלפים מצליח לעבור.

יעל: האשמה המיידית תהיה שה-PLC נהיה איטי מדי.

אמיר: נכון, אבל "איטי מדי" עדיין אינו אבחון הנדסי. האם הקלט נדגם באיחור? האם Task בעדיפות נמוכה עיכב את ההחלטה? האם תמונת הפלט עודכנה רק בסוף מחזור אחר? האם הרשת הוסיפה Jitter? האם השסתום נע לאט יותר בגלל ירידת לחץ אוויר? או שהמכניקה פשוט תוכננה עם פחות מדי מרווח?

יעל: כלומר מהירות המעבד לבדה לא אומרת אם המכונה מגיבה בזמן.

אמיר: בדיוק. זמן התגובה הוא שרשרת מקצה לקצה: אירוע, חיישן, מודול קלט, תקשורת, תזמון Task, לוגיקה, עדכון פלט, מפעיל ומכניקה. היום נפתח את הבקר ונבין איך השרשרת הזו מאורגנת.

יעל: וכשאנחנו אומרים "נפתח", הכוונה מושגית. לא עובדים בארון חי בלי נוהל מתאים.

אמיר: Interlock ראשון מצוין לפרק.

02:30-05:30 | מה נחשב בקר תעשייתי?

יעל: בשרטוטים אני רואה IPC, PAC, PLC, בקר DCS, בקר Motion ובקר בטיחות. האם אלה מחשבים שונים לגמרי?

אמיר: לפעמים החומרה דומה. השאלה המועילה יותר היא אילו הבטחות ביצוע, ממשקים, תנאי סביבה ומחזור חיים המוצר נועד לספק. PLC הוא בקר תעשייתי דיגיטלי שמבצע לוגיקה של המשתמש ושולט במכונות או בתהליכים באמצעות קלטים ופלטטים. IEC 61131-1 נותן את המסגרת הכללית, ו-IEC 61131-2 עוסק בדרישות פונקציונליות, תאימות אלקטרומגנטית ובדיקות לציוד בקרה תעשייתי, כולל PLC ו-PAC.

יעל: זה כבר אומר שבקר תעשייתי אינו סתם מחשב משרדי קטן בקופסת מתכת.

אמיר: נכון. הוא מתוכנן סביב מתח תעשייתי, טמפרטורה, רעידות, הפרעות אלקטרומגנטיות, דיאגנוסטיקה של מודולים, התנעה מבוקרת ותמיכה ארוכת שנים. PAC הוא מונח נפוץ לפלטפורמות עם יותר זיכרון, רשתות עשירות, Multi-tasking, Motion, ספריות תהליך או יתירות. אין קו אוניברסלי שבו PLC הופך ל-PAC. צריך לפרק את השם ליכולות מדידות.

יעל: ומהו IPC?

אמיר: Industrial PC הוא מחשב מוקשח לסביבה תעשייתית. הוא יכול להריץ Windows, Linux, מערכת זמן אמת ו־Runtime ייעודי. ב-Soft PLC, סביבת הבקרה היא תוכנה שרצה על המחשב. אפשר לשלב על אותה פלטפורמה לוגיקת PLC, Motion, ראייה ממוחשבת, אנליטיקה ו-HMI. השאלה היא כיצד עומס זמן האמת מבודד מתוכנות אחרות, כיצד מנהלים אחסון ועדכונים, ומה קורה אם מערכת ההפעלה המארכת נכשלת.

יעל: בקר DCS נבחר כחלק ממערכת שלמה ולא רק כקופסה.

אמיר: בדיוק. DCS מביא בדרך כלל בסיס נתונים הנדסי משותף, ספריות תהליך, ניהול אזהרות, קישור ל-Historian ודפוסי יתירות ברמת המפעל. Safety PLC שונה שוב: הוא מיועד להיות תת-מערכת הלוגית במערכת בטיחות ונדרש למחזור חיים, כלים ודיאגנוסטיקה מוגדרים. מארז דומה, טענה הנדסית אחרת.

יעל: כלל ראשון: להתעלם לרגע מהלוגו ולכתוב דרישות.

אמיר: זמן תגובה, I/O, סביבה, זמינות, תקשורת, תפקיד בטיחותי, סייבר, תחזוקתיות ואורך חיים. רק אז משווים ארכיטקטורות.

05:30-10:00 | בתוך חומרת הבקר

יעל: נוריד בדמיון את דלת הארון. מה יש בתוך Rack מודולרי?

אמיר: בדרך כלל ספק כוח, CPU או מודול בקר, מודולי קלט ופלט מקומיים ומודולי תקשורת או מודולים מיוחדים. Base או Backplane מחברים ביניהם ומעבירים מתח ונתונים. בבקרים קומפקטיים רוב הרכיבים נמצאים באותו מארז. בפלטפורמות גדולות הם מפוזרים בין Racks ותחנות I/O Remote.

יעל: ספק הכוח נשמע רכיב פשוט, אבל הוא קובע התנהגות.

אמיר: בהחלט. צריך לשאול כמה זמן הבקר ממשיך לפעול בזמן נפילת מתח קצרה, איך מזהה Brownout, איזו דיאגנוסטיקה נוצרת, והאם יש תחומי מתח נפרדים ללוגיקה, לקלטים ולפלטטים. ה-CPU יכול להמשיך לרוץ בזמן שאספקת הפלטטים נפלה. אם התוכנה מסתכלת רק על Command, ה-HMI עלול להציג "שסתום פתוח" כאשר לסולנואיד אין מתח.

יעל: ואז ה-CPU.

אמיר: ה-CPU מריץ את ה-Runtime, מתזמן Tasks, מנהל זיכרון, דיאגנוסטיקה ותקשורת. במפרט יופיעו ביצועי הוראות, זיכרון אפליקציה, קיבולת תקשורת, מספר צירים, מגבלות Tasks ויכולות אבטחה. לא משווים בקרים לפי מספר Benchmark אחד. לוגיקה בוליאנית, חישובי נקודה צפה, Motion, תקשורת ועיבוד נתונים מעמיסים משאבים שונים.

יעל: גם הזיכרון אינו דלי אחד.

אמיר: נכון. ייתכנו Load Memory לפרויקט, Work Memory לביצוע, אחסון בלתי נדיף, אזורי Retentive, Buffers ומדיה נשלפת. השמות משתנים לפי יצרן, אך השאלות קבועות: מה שורד אובדן מתח, מה שורד Warm Restart, מה מאותחל מחדש ומה צריך להיטען מ-Recipe או ממערכת עליונה?

יעל: גם מודול תקשורת יכול לכלול מעבד.

אמיר: כן. יש Ethernet ו-Fieldbus משולבים, ומודולים נפרדים ל-Serial, Motion, Safety או Remote I/O. חלק מהתקשורת מחזורית ומתוזמנת היטב, וחלק Explicit או Acyclic. חלק מהעבודה נעשית ב-CPU וחלק בחומרה ייעודית. לכן Polling אגרסיבי של HMI או העברת קבצים עלולים להתחרות בעומסים אחרים אם אין הפרדה.

יעל: ומה לגבי מודולים מיוחדים?

אמיר: High-speed Counter, ממשק Encoder, קלטים עם Timestamp, Thermocouple, Strain Gauge וממשקי Motion. הם מבצעים לכידה או המרה בחומרה, משום ש-Task ראשי של עשר מילישניות לא יכול להבטיח שיזהה אירוע של מיקרו-שניות.

יעל: כלומר Rack הוא מערכת מחשוב מבוצרת קטנה, לא לוח מהדקים.

אמיר: זו דרך מצוינת לראות אותו.

Scan Cycle, Tasks | 10:00-15:30 ועדיפויות

יעל: בכל קורס בסיסי מציגים Scan: קריאת קלטים, ביצוע תוכנה, עדכון פלטים וחוזר חלילה. זה עדיין נכון?

אמיר: זה מודל פתיחה טוב, לא תיאור מלא. בקרים רבים מחזיקים Input Image, מריצים לוגיקה ומעדכנים Output Image. אבל פלטפורמות מודרניות מפעילות כמה Tasks: Continuous, Periodic, Event-driven או מסונכרנים ל-I/O Bus ול-Motion. ב-Rockwell הביצוע מאורגן דרך Tasks, Programs ו-Routines. ב-CODESYS אפשר להגדיר Tasks מחזוריים, Freewheeling ומונעי אירוע, עם עדיפויות ו-Watchdog. המילים משתנות, העיקרון הוא Scheduling.

יעל: תסביר Task מחזורי.

אמיר: נניח שהוא מוגדר כל עשר מילישניות. ה-Scheduler משחרר אותו במרווח הזה. הוא מריץ את התוכניות שלו. אם הביצוע לוקח שתי מילישניות, נשאר זמן למשימות אחרות ולשירותי מערכת. אם לעיתים הוא דורש שתי-עשרה מילישניות, הוא החמיץ את המחזור המתוכנן. בהתאם ל-Runtime הוא יכול לעכב את המחזור הבא, לייצר דיאגנוסטיקה או להפעיל Watchdog.

יעל: Continuous Task רץ כשאינו משהו חשוב יותר?

אמיר: לעיתים כן, אך המימוש משתנה. Event Task מופעל בגלל אירוע חומרה או תוכנה: סימן Registration, קלט מהיר, אירוע תקשורת או תנאי פנימי. זה מקטין Latency, אבל הופך את האינטראקציה בין Tasks למורכבת יותר. שטף אירועים יכול להרעיל Tasks נמוכים אם התכן לא זהיר.

יעל: ועדיפויות. בפלטפורמה אחת 1 היא גבוהה ובאחרת 0.

אמיר: לכן לא מסיקים מהמספר בלי תיעוד. ברמה העקרונית Task גבוה יכול לבצע Pre-emption ל-Task נמוך. כך Motion מהיר או פונקציה קריטית עומדים ב-Deadline. אבל נתונים משותפים דורשים משמעת. אם Task איטי מעדכן Task-1 Structure מהיר קורא באמצע, הוא עלול לראות Snapshot לא עקבי.

יעל: איך מונעים זאת?

אמיר: משתמשים במנגנוני העברת נתונים של הפלטפורמה, מעתיקים מבנים שלמים בנקודות מוגדרות, מגדירים בעלות ברורה ונמנעים מכתובה של כמה Tasks לאותו נתון. יש מערכות עם Double Buffer, Producer-Consumer, מנגנוני נעילה או I/O מסונכרן. הכלי משתנה; העיקרון הוא בעלות והעברה דטרמיניסטיות.

יעל: איפה Input Image נכנס לתמונה?

אמיר: הוא נותן לתוכנית קבוצת קלטים עקבית לחלון ביצוע. אך יש מערכות שבהן I/O מתעדכן Asynchronous, יש הוראות Immediate ו-I/O Remote יש לוח זמנים משלו. אם הלוגיקה מניחה שכל Tags השתנו באותה נקודת זמן, ייתכן שההנחה שגויה.

יעל: ה-Scan הוא מפה, ולוח ה-Tasks הוא לוח הזמנים האמיתי.

אמיר: בדיוק. והוא צריך להיות חלק מהתכן הפונקציונלי, לא משהו שמגלים בזמן תקלה.

Watchdog ו-Jitter | דטרמיניזם, 15:30-19:30

יעל: כשאתה אומר דטרמיניסטי, האם כל מחזור חייב לקחת בדיוק אותו מספר מיקרו-שניות?

אמיר: לא. המשמעות היא שהזמנים חסומים וצפויים מספיק לפונקציה. חוג טמפרטורה יכול לסבול עשרות או מאות מילישניות. Electronic Camming דורש סנכרון הדוק בהרבה. הדרישה צריכה להגדיר זמן תגובה או Jitter מרבי, לא רק "מהיר".

יעל: נבנה מחדש את שרשרת שער הפסילה.

אמיר: לחיישן יש זמן תגובה. מודול הקלט דוגם ומסנן. רשת Remote I/O מעבירה. ה-Task מחכה לשחרור הבא, מבצע לוגיקה ומעדכן פלט. מודול הפלט מחליף מצב. הסולנואיד והצילינדר נעים. זמן התגובה הגרוע ביותר הוא סכום התרומות הגרועות ביותר והאי-ודאות. ממוצע מסתיר את המקרים המאוחרים שגורמים לפסילה להיכשל.

יעל: Jitter הוא השונות סביב הזמן הצפוי.

אמיר: נכון: שונות בהתחלת Task, בזמן הביצוע, ברשת ובמפעיל. כדאי לרשום Min, Average ו-Max ולא להסתפק בצילום רגעי. גם עומס CPU צריך Margin. ארבעים אחוז ממוצע לא מבטיח שאין Burst שחוסם חלון של Task גבוה.

יעל: מה עושה Watchdog?

אמיר: הוא בודק ש-Task או בקר משלימים עבודה בתוך זמן מוגדר. חריגה יכולה ליצור Fault, לעצור Task, לעצור אפליקציה או להעביר פלטים למצבים מוגדרים. תיעוד CODESYS, למשל, מתאר זמן Watchdog ו-Sensitivity. ההגדרה צריכה להיות מעל זמן הביצוע הלגיטימי הגרוע ביותר, ומתחת לזמן שבו אובדן השליטה כבר אינו מקובל.

יעל: הגדרה צרה מדי תגרום Trips בהקמה. רחבה מדי תזהה את הכשל אחרי שהתהליך כבר בסיכון.

אמיר: בדיוק. Watchdog אינו תחליף לפונקציית בטיחות עצמאית. הוא מגן על שלמות הביצוע. עדיין צריך להגדיר מה קורה לפלטים: מי יורד, מי נשאר, איזה ציוד דורש עצירה מבוקרת ואילו שכבות עצמאיות ממשיכות להגן.

יעל: כאן תזמון תוכנה נפגש עם אנרגיה מכנית אגורה.

אמיר: לחץ, אינרציה, כבידה, חום וכימיה אינם עוצרים מפני שה-CPU נכשל.

19:30-24:00 | I/O מקומי, Remote I/O ומסלול האות

יעל: למה לבחור Remote I/O במקום להביא כל כבל לארון הראשי?

אמיר: Remote I/O יכול לקצר כבלים, לפשט בנייה מודולרית ולמקם דיאגנוסטיקה ליד הצידוד. הוא גם יוצר גבול טבעי למודול מכונה. אבל הוא מוסיף רשת, חלוקת מתח ומצב כשל תקשורתי. הבחירה אינה "מרוחק הוא מודרני", אלא איזון בין חיווט, תחזוקה, סביבה, Latency, טופולוגיה וזמינות.

יעל: מה הבקר יודע כש-Rack מרוחק נעלם?

אמיר: זה תלוי בפלטפורמה ובהגדרה. קלטים יכולים לקבל Quality Bad, להחזיק ערך אחרון או לעבור לערך חלופי. פלטים יכולים להחזיק, לרדת לאפס או לקבל Fallback. התוכנה חייבת להשתמש ב-Quality וב-Connection Status ולא להתייחס לערך ישן כאמין. ספיקה של ארבעים יכולה להיות "נמדדה עכשיו" או "הערך האחרון לפני אובדן תקשורת". אלה שני מצבי תהליך שונים.

יעל: Digital I/O נשמע פשוט יותר.

אמיר: פשוט יותר, לא פשוט. סוג קלט, Source או Threshold, Sink, סינון, בידוד, זיהוי קצר ו-Test Pulses חשובים. פלט יכול להיות Relay, Transistor או Triac. Leakage Current יכול להשאיר עומס רגיש מופעל חלקית. מודול יכול לדווח שהפקודה פעילה בלי להוכיח זרם שדה או תנועה. לפעולה קריטית צריך Feedback עצמאי שמתאים לכשל שרוצים לגלות.

יעל: באנלוגי מתווספים זמני המרה ודיוק.

אמיר: וגם קונפיגורציה, Wire Break, Common Mode, בידוד, Update Rate-1 Resolution, Filter. מספר של Bit 16 אינו מבטיח דיוק מערכת של Bit 16. אי-ודאות החיישן, החיווט, הייחוס, המודול וה-Scaling מצטברות. זה יהיה לב פרק 3.

יעל: ומה עם Timestamps?

אמיר: לניתוח Sequence of Events צריך ללכוד זמן קרוב מספיק לאירוע. אם ה-HMI נותן Timestamp לאחר Polling, שני אירועים יכולים להופיע בסדר שגוי. יש מודולים שקולטים Timestamp בקצה ויש בקרים שמסנכרנים שעונים ברשת. הארכיטקטורה נגזרת מדרישת האבחון.

יעל: רשימת I/O צריכה לכלול יותר מ-Tag וטווח.

אמיר: כן: סוג חשמלי, זמן עדכון, מצב כשל, דיאגנוסטיקה, בידוד, סביבה, מקור מתח, הנחות כבל והארקה, ואיך התוכנה משתמשת ב-Quality.

24:00-28:00 | זיכרון, Retention והתנעה מחדש

יעל: נניח שהחשמל נופל באמצע מילוי. מה הבקר צריך לזכור?

אמיר: צריך להחליט משתנה-משתנה. שמירת הכול נשמעת בטוחה, אך יכולה לשמר מצב לא תקף. מחיקת הכול יכולה לאבד כיוול, ספירת ייצור או מיקום חומר. מסווגים נתונים לפי מטרה ולפי תרחיש Restart.

יעל: תן קטגוריות.

אמיר: קונפיגורציה וכיוול דורשים בדרך כלל אחסון בלתי נדיף מבוקר. Recipes יכולים להישמר מקומית או להיטען ממערכת מתכונים. Counters עשויים לדרוש שמירה עם אסטרטגיית כתיבה מוגדרת. חישובים זמניים צריכים לרוב

להתאפס. מצב Sequence הוא הקטגוריה המסוכנת: לאחר מתח, האם חוזרים אוטומטית לשלב 17, עוברים למצב Recovery בטוח, או דורשים בדיקה ופינוי חומר?

יעל: במכונת המילוי חזרה אוטומטית יכולה למלא פעמיים או להזיז שולחן בזמן שמישהו מנקה.

אמיר: נכון. הבקר יכול לשמור Position, אבל המכניקה אולי זה לא מתח. Absolute Encoder, חיישן Home וערך תוכנה Retentive הם שלושה מקורות אמת שונים. לפני חזרה לפקודות צריך לאמת מצב פיזיקלי.

יעל: מה ההבדל בין Warm ל-Cold Restart?

אמיר: המונחים משתנים בין יצרנים, אך בדרך כלל Warm משמר יותר מצב Cold-1 Runtime מאתחל יותר. אסור להסתמך על המילה בלבד. בודקים בפועל Power Loss, Stop-Run, עדכון Firmware, Download של פרויקט וכשל סוללה או אחסון. מתעדים מה נשמר בכל מצב.

יעל: גם לזיכרון בלתי נדיף יש מגבלות כתיבה.

אמיר: לעיתים. הפלטפורמות מנהלות זאת אחרת, אבל כתיבה ל-Flash בכל Scan היא הנחה רעה. משתמשים במנגנוני Retain נתמכים, Buffer וכתיבה Transactional. עבור רשומות חשובות צריך לשאול אם ה-PLC בכלל צריך להיות בסיס הנתונים הראשי. הוא מצוין בבקרה; הוא לא בהכרח Historian ארוך טווח.

יעל: צריך לבדוק Restart ב-FAT.

אמיר: בהחלט. מנתקים מתח בשלבים מוגדרים, מחזירים ובודקים פלטים, ערכים שמורים, אזעקות, Audit והנחיות למפעיל. Restart הוא מצב עבודה מתוכנן, לא תאונה בין Run ל-Stop.

IEC 61131-3 | 28:00-32:00 וסביבת הפיתוח

יעל: מה בדיוק IEC 61131-3 מתקן?

אמיר: הוא מגדיר Syntax ו-Semantics למשפחת שפות ומושגי קונפיגורציה לבקרים. המהדורה הרביעית פורסמה ב-2025. התקציר הרשמי של IEC מונה Function Block Diagram, Structured Text, Ladder Diagram, Function Block Diagram, יחד עם רכיבי Sequential Function Chart לארגון תוכניות ו-Function Blocks. במערכות מותקנות עדיין קיימים פרויקטים וכלים עם Instruction List, ולכן צריך להבחין בין התקן העדכני למציאות ה-Legacy.

יעל: ומהם Program Organization Units?

אמיר: Function Blocks-1 Programs, Functions הם יחידות מבנה מרכזיות. Function מחזיר תוצאה בלי מצב פנימי מתמשך של Instance. Function Block מחזיק נתוני Instance ומתאים למנוע, שסתום, PID או מודול ציוד חוזר. Program מארגן ביצוע אפליקציה. יצרנים מוסיפים Namespaces-1 Libraries, Classes, Methods בהתאם למהדורה ולפלטפורמה.

יעל: האם התקן אומר שאפשר להעביר פרויקט מכל יצרן לכל יצרן?

אמיר: לא. הוא הופך מושגים ושפות למוכרים יותר, אך קונפיגורציית חומרה, Libraries, Data Types, Task Model, Motion, Safety ודיאגנוסטיקה שונים. IEC 61131-10 מגדיר פורמט XML להחלפת פרויקטים ו-PLCopen מפתח Conformance ומפרטים לשימוש חוזר, אך ניידות מעשית עדיין דורשת הנדסה ובדיקות.

יעל: סביבת הפיתוח מאפשרת Online Monitoring ושינוי תוך כדי עבודה.

אמיר: וזו יכולת חזקה מאוד. אפשר לצפות בערכים, לבצע Force, לשנות קוד ולהוריד גרסה. כל יכולת כזו צריכה הרשאה, ניהול שינוי והתאוששות. Online Change עשוי לשמר מצב תהליך, אך הוא גם יכול ליצור קונפיגורציה שלא עברה את אותו FAT כמו ה-Baseline המאושר.

יעל: אז Version Control אינו קובץ בשם final-final-2.

אמיר: נכון. שומרים Source, Libraries, Hardware Configuration, תלות HMI, תאימות Firmware, Safety Signatures אם נדרש ו-Backup שאפשר לפרוס. מתעדים מי שינה, למה, מי סקר ואילו בדיקות בוצעו. הנחיות PLCopen לבניית תוכנה שימושיות משום שגם תוכנת בקרה סובלת מבעיות תחזוקתיות, רק עם תוצאה פיזיקלית. יעל: בחירת השפה עצמה תגיע בהמשך?

אמיר: כן. היום המסר הוא שהשפה רצה בתוך Runtime מתוזמן, עם חומרה ומחזור חיים. Structured Text אלגנטי. בתוך Task שגוי הוא עדיין תכן בקרה גרוע.

32:00-36:00 | בחירה בין PLC, PAC ו-IPC

יעל: נבחר בקר למכונת המילוי שלנו. יש שלושים צירי Servo, ראייה ממוחשבת, שמונים נקודות אנלוגיות, Recipes, HMI וחיבור למפעל. PLC, PAC או IPC?

אמיר: אני מסרב לענות לפני שמפרקים עומסים. כמה צירים דורשים Motion מתואם? מה Cycle Time והסנכרון? האם מערכת הראייה מחזירה Pass-Fail או עושה עיבוד כבד? מה זמני החוגים האנלוגיים? איזו זמינות נדרשת? כמה שנים תתוחזק המכונה? מי מטפל בה בשתיים בלילה?

יעל: נניח שעיבוד התמונה כבד וה-Motion מסונכרן מאוד.

אמיר: פלטפורמת PC-based יכולה להתאים, משום שאפשר להקצות Cores לזמן אמת ואחרים לראייה או אנליטיקה. Beckhoff, למשל, מתארת TwinCAT כמערכת PC-based שבה זמן אמת רץ בנפרד ממערכת ההפעלה ויכול להשתמש בכמה ליבות. זו דוגמת מימוש, לא הבטחה אוניברסלית. עדיין נבדוק Worst Case, התאוששות מאחסון, Patch Strategy והפרדה בין עומסים.

יעל: גם PLC או PAC מתקדם יכול לשלב Motion ותהליך.

אמיר: כן. הוא יכול לתת מודל תחזוקה מוכר, I/O תעשייתי משולב ומחזור חיים ארוך. אם לבונה המכונה יש Libraries מוכחות ויכולת שירות בפלטפורמה הזו, הסיכון הכולל עשוי להיות נמוך יותר, גם אם המחשב פחות פתוח. זו החלטת Lifecycle, לא תחרות מעבדים.

יעל: אפשר לפצל: IPC ל-Motion וראייה, PLC לתהליך ו-Utilities.

אמיר: אפשר, אך כל פיצול יוצר Interfaces, סנכרון ובעלות. מי מחזיק Machine State? מה קורה אם בקר אחד עושה Restart? איך מסנכרנים Recipe? האם כשל רשת יוצר פקודות סותרות? ארכיטקטורה מבוססת טובה כשהיא יוצרת מודולריות או Fault Containment, לא רק מפזרת קוד.

יעל: ומה עושים עם המילה PAC?

אמיר: מתרגמים אותה לדרישות: כמה Tasks דטרמיניסטיים, זיכרון, Motion, Ethernet, Structured Data, ספריות Process, יתירות, Safety או Protocols. ברכש משווים פונקציות מוכחות, לא תארים.

36:00-39:30 | יתירות וזמינות גבוהה

יעל: הנהלה מבקשת PLC כפול כי השבתה יקרה. מה שואלים?

אמיר: מה בדיוק כפול? CPU, ספק כוח, רשת, Remote I/O Adapter, Server, תחנת הנדסה, חיישן, שסתום או Utility? שני בקרים אינם מבטלים הזנת מתח יחידה, Switch אחד או שסתום יחיד.

יעל: איך עובד Hot Standby ברמה העקרונית?

אמיר: בקר אחד Primary ומריץ את האפליקציה. ה-Standby מקבל מצב מסונכרן ומנטר בריאות. בכשל מתאים השליטה עוברת. תיעוד M580 של Schneider, לדוגמה, מתאר שני בקרים בקונפיגורציה זוהי, ניטור רציף ועדכון

מצב מה-Primary ל-Standby. אבל זמן מעבר, הנתונים שמסונכרנים, טופולוגיית I/O והתנהגות ב-Mismatch הם ספציפיים לפלטפורמה.

יעל: המעבר תמיד Bumpless?

אמיר: לא מניחים זאת בלי להגדיר איזה משתנה ומה גבול הקבלה. CPU יכול לעבור בתוך Scan אחד בזמן שחיבור תקשורת מתאושש לאט יותר. Analog Output יכול להחזיק, לקפוץ או להיות מחושב מחדש ממצב מעט שונה. Motion יכול לדרוש Re-synchronization מבוקר ולא מעבר שקוף.

יעל: אילו בדיקות כשל שייכות ל-FAT או SAT?

אמיר: אובדן Primary, אובדן Standby, ספק כוח, נתיב רשת, Rack מרוחק, Clock Sync, חיבור Engineering ומודולי תקשורת נבחרים. גם Configuration Mismatch, כשל Switchover, Switchback והחלפה בזמן תחזוקה. מודדים השפעת תהליך, רצף אזהרות ופעולת התאוששות. זמינות גבוהה מוכחת בבדיקת כשל, לא בצירוף.

יעל: ומה לגבי Common Cause?

אמיר: שני בקרים זהים יכולים לשתף באג תוכנה, סביבה, Credential או קונפיגורציה שגויה. יתירות משפרת זמינות מול כשלים נבחרים; היא אינה הופכת את המערכת לחסינה.

39:30-42:00 | גבולות בטיחות וסייבר

יעל: האם PLC רגיל יכול לעצור הכול בבטחה בכל תקלה?

אמיר: הוא יכול לבצע Interlocks רגילים ועצירות מבוקרות, אך פונקציית בטיחות דורשת ארכיטקטורה ומחזור חיים מבוססי סיכון. IEC 61131-6 מגדיר דרישות ל-FS-PLC המיועד להיות תת-המערכת הלוגית במערכת בטיחות חשמלית או אלקטרונית. זה לא הופך כל תוכנה בבקר בטיחות לבטוחה. חיישנים, לוגיקה, Final Elements, דיאגנוסטיקה, עצמאות, Proof Testing-1 Validation חשובים כולם.

יעל: אפשר להריץ לוגיקה רגילה ובטיחותית על אותה פלטפורמה?

אמיר: יש פלטפורמות מאושרות שמאפשרות זאת עם מנגנוני הפרדה, אבל אפליקציית הבטיחות עדיין כפופה לתקן המכונה או התהליך ול-Safety Manual של היצרן. נוחות אינה ראייה לעצמאות.

יעל: סייבר הוא שכבה סביב הבקר.

אמיר: וגם בתוכו. NIST SP 800-82 Rev. 3 מדגיש אבטחת OT תוך שמירה על ביצועים, אמינות ובטיחות. ISA/IEC 62443 מחלק אחריות בין Asset Owner, אינטגרטור, ספק שירות ויצרן מוצר. ברמת הבקר מדובר במלאי נכסים, Segmentation, חשבונות לפי תפקיד, ביטול Default Credentials, גישה מרחוק מאובטחת, Firmware מבוקר, גיבויים, Logging והשבתת שירותים לא נחוצים כאשר הפלטפורמה תומכת.

יעל: ומה עם Online Change-1 Programming Port?

אמיר: מתייחסים אליהם כממשקי תחזוקה רבי עוצמה. מאמתים משתמשים, מגבילים נתיבי רשת, משתמשים בתחנות Engineering מאושרות, שומרים עותק Recovery Offline ומנטרים שינויים. בקר שמפקד על מנועים ושסתומים אינו רק Data Endpoint. פגיעה בו יכולה להפוך למומנט, לחץ, חום או שחרור כימי.

יעל: אבל גם לא מעדכנים Patch בעיוורון.

אמיר: נכון. סייבר OT דורש חלון שינוי בדוק ותוכנית התאוששות. "לעולם לא לעדכן" צובר סיכון; "לעדכן מיד בלי Validation" יכול ליצור סיכון אחר. מחזור החיים צריך לאזן פגיעות, זמינות ותפעול מאומת.

42:00-44:00 | מפת Tasks וצ'קליסט סיום

יעל: נסיים במפת Tasks להמחשה עבור מכונת המילוי.

אמיר: ארכיטקטורה אפשרית יכולה לכלול Motion Task של מילישנייה המסונכרן לרשת ה- Drives, Machine Sequence של עשר מילישניות, Process Control של חמישים מילישניות לספיקה וטמפרטורה, Diagnostics של מאה מילישניות ושירותי HMI ונתונים איטיים יותר. אלה דוגמאות בלבד. הערכים האמיתיים נגזרים מפיזיקת התנועה, דינמיקת החיישנים והמפעילים, תזמון הרשת וזמן הביצוע הנמדד.

יעל: הבעלות על נתונים ברורה. Motion מחזיק מצב צירים. Sequence מחזיק Machine Mode. Process Control מחזיק Validation לפני שהן הופכות לפקודות. מחזיק יציאות חוג. בקשות HMI עוברות Validation לפני שהן הופכות לפקודות.

אמיר: בדיוק. לכל Task מגדירים Period, Priority, Worst-case Execution, Watchdog, תקובת Watchdog, מקורות קלט, בעלות על פלט ושיטת בדיקה. Restart מוגדר. לכשל I/O יש Remote I/O Feedback. Fallback פיזיקלי קריטי אינו מוחלף ב-Command Bit.

יעל: תן את שבע השאלות לפני בחירת בקר.

אמיר: אחת: אילו פונקציות פיזיקליות וזמני תגובה גרועים ביותר נדרשים? שתיים: אילו סוגי I/O, קצבי עדכון, דיאגנוסטיקה ודירוגי סביבה? שלוש: איזה Task Model, Motion, Process ותקשורת נדרשים עם Margin? ארבע: אילו נתונים צריכים לשרוד כל מצב Restart? חמש: איזו ארכיטקטורת זמינות ואילו בדיקות כשל? שש: אילו אחריות Safety וסייבר נמצאות בתוך הבקר ומחוצה לו? שבע: מי יתחזק את המערכת, באילו כלים, גיבויים, מיומנויות ותמיכת יצרן לאורך החיים?

יעל: כך השאלה משתנה מ-"איזה PLC הכי מהיר?" ל-"איזו ארכיטקטורה יכולה להוכיח את ההתנהגות הנדרשת?"

אמיר: בדיוק. בפרק 3 נצא צעד אחד החוצה אל החיישנים: כיצד לחץ, ספיקה, טמפרטורה, מיקום ונוכחות הופכים לנתון אמין, וכיצד חיווט, Sampling, Calibration ומצבי כשל יכולים להטעות גם קוד מושלם.

יעל: עד אז זכרו: הבקר אינו רואה את המכונה. הוא רואה אותות.

אמיר: והאיכות, התזמון והמשמעות שלהם קובעים את איכות הבקרה. תודה שהאזנתם ל-Control Edge.

יעל: נתראה בפרק 3.

7. הערות למפיק ולמגישים

- לשמור על קצב מקצועי ורגוע. להשאיר השהיה אחרי הסבר זמן התגובה מקצה לקצה ואחרי דוגמת ההתנהגות מחדש.
- לא להפוך את דוגמאות היצרנים לדירוג. המטרה היא להראות שמושגים משותפים מיושמים אחרת בפלטפורמות שונות.
- בכל שימוש במונח Scan Cycle להבהיר שמערכות מודרניות עשויות להפעיל כמה Tasks מחזוריים ומונעי אירוע.
- להשתמש במונח "זמן אמת" במובן של תזמון חסום וצפוי מספיק ליישום, ולא רק מעבד מהיר.
- זמני ה-Task בדוגמה הם המחשה בלבד ואינם ערכים אוניברסליים.
- בהתייחסות ל-IEC 61131-3:2025 לציין שמדובר במהדורה הרביעית ולהימנע מהבטחה לניידות מלאה בין סביבות פיתוח.
- דיון ה-Safety PLC נשאר ארכיטקטוני. חישובי PL/SIL ומחזור החיים המלא יופיעו בהמשך הסדרה.

8. צ'קליסט תכן לבחירת בקר

1. מהן הפונקציות הפיזיקליות, זמני התגובה המרביים וה-Deadlines?

2. אילו סוגי I/O, זמני עדכון, דיאגנוסטיקה, בידוד ותנאי סביבה נדרשים?
3. מהו מודל ה-Tasks, ה-Priority, ה-Watchdog וה-Margin של ה-CPU?
4. אילו נתונים נשמרים בכל Cold Restart, Power Loss, Stop-Run, Warm Restart?
5. מהי ארכיטקטורת הזמניות ואילו נקודות כשל נשארות יחידות?
6. אילו פונקציות שייכות לבקרה רגילה, Safety ולשכבות הגנה עצמאיות?
7. כיצד מנהלים משתמשים, גישה מרחוק, Change Management, Firmware, Backup, Version Control?
8. מהי אסטרטגיית תחזוקה, חלפים, רישוי, תמיכת יצרן והעברת ידע לעשר עד עשרים שנה?

9. מילון מונחים

מונח	משמעות בפרק
Backplane	התשתית החשמלית והתקשורתית המחוברת בקר, ספק כוח ומודולים בתוך Rack או Base.
Cycle Time	הזמן לביצוע מחזור Task או מחזור בקר; אינו בהכרח זמן התגובה הפיזיקלי הכולל.
דטרמיניזם	התנהגות זמנים חסומה וצפויה מספיק עבור פונקציית הבקרה.
Event Task	לוגיקה שמופעלת בעקבות אירוע מוגדר, לפי מודל התזמון של הבקר.
FS-PLC	בקר לוגי מתוכנת לבטיחות פונקציונלית, המיועד לשמש כתת-המערכת הלוגית של מערכת בטיחות.
מחשב תעשייתי - IPC	פלטפורמת מחשב מוקשחת לסביבה תעשייתית, שלעיתים מריצה Runtime בקרה בזמן אמת.
Jitter	שונות בזמן ההתחלה או משך הביצוע של Task מחזורי או עדכון תקשורת.
PAC	Programmable Automation Controller - מונח נפוץ בתעשייה, אך לא גבול ארכיטקטוני אוניברסלי קשיח.
POU	Program Organization Unit ב-IEC 61131-3, כגון Program, Function או Function Block.
Retentive Data	נתונים שנשמרים בכוונה לאורך מצב Restart או אובדן מתח שהוגדר מראש.
Scan Cycle	מודל מפושט שבו נקראים קלטים, מבוצעת לוגיקה

משמעות בפרק	מונח
ומתעדכנים פלטים; בפועל ייתכנו כמה Tasks ומנגנוני I/O.	
Runtime של PLC הממומש בתוכנה על מחשב כללי או תעשייתי.	Soft PLC
מנגנון תזמון שמזהה Task או בקר שלא השלימו ביצוע בתוך גבול מוגדר.	Watchdog

10. מפת תקנים ומקורות

איך להשתמש ברשימה

תקנים מתעדכנים ומאומצים באופן שונה לפי מדינה, ענף וחווה. יש לרכוש או לגשת למהדורה המחייבת בפרויקט ולבדוק תיקונים, אימוץ לאומי והוראות יצרן.

IEC 61131-1:2003 - מידע כללי על בקרים מתוכנתים

הגדרות ומאפיינים פונקציונליים מרכזיים של מערכות PLC. [מקור רשמי](#)

IEC 61131-2:2017 - דרישות ובדיקות לציוד בקרה תעשייתי

דרישות פונקציונליות ו-EMC ובדיקות אימות לציוד בקרה, כולל PLC ו-PAC. [מקור רשמי](#)

IEC 61131-3:2025 - שפות תכנות לבקרים

המהדורה הרביעית של תקן השפות, שפורסמה ב-22 במאי 2025. [מקור רשמי](#)

IEC 61131-6:2012 - בטיחות פונקציונלית

דרישות ל-FS-PLC המשמש כתת-המערכת הלוגית של מערכת בטיחות. [מקור רשמי](#)

IEC 61010-2-201:2024 - דרישות מיוחדות לציוד בקרה

דרישות בטיחות חשמליות לציוד בקרה בשילוב IEC 61010-1. [מקור רשמי](#)

IEC 61131-10:2019 - פורמט PLCopen XML

פורמט XML להחלפת פרויקטי IEC 61131-3; אינו מבטיח ניידות ללא עבודת הנדסה. [מקור רשמי](#)

PLCopen Software Construction Guidelines

כללי קוד, ספריות, OOP, SFC ומדדי איכות לתוכנת בקרה תעשייתית. [מקור רשמי](#)

CODESYS - Task Configuration

תיאור רשמי של Tasks מחזוריים, Freewheeling ומונעי אירוע, עדיפויות ו-Watchdog. [מקור רשמי](#)

Rockwell Studio 5000 - Tasks, Programs and Routines

מושגי ארגון ביצוע רשמיים לבקרי Logix. [מקור רשמי](#)

Beckhoff TwinCAT - PC-based Control

סקירה רשמית של PLC, Motion ו-Runtimes בזמן אמת על מחשב תעשייתי. [מקור רשמי](#)

Schneider Electric Modicon M580 Hardware Reference Manual

[מקור רשמי](#) לארכיטקטורת I/O-1 Controller, Tasks ומרוחק. [מקור רשמי](#)

Schneider Electric M580 Hot Standby System Guide

[מקור רשמי](#) דוגמה רשמית לסנכרון מצב ולארכיטקטורת זמינות גבוהה. [מקור רשמי](#)

OPC UA for PLCs based on IEC 61131-3

[מקור רשמי](#) מודל מידע משותף של OPC Foundation I-PLCopen לאינטגרציית בקרים. [מקור רשמי](#)

NIST SP 800-82 Rev. 3

[מקור רשמי](#) הנחיות סייבר ל-OT תוך התייחסות לביצועים, אמינות ובטיחות. [מקור רשמי](#)

ISA/IEC 62443 סדרת

[מקור רשמי](#) מחזור חיים ודרישות סייבר לבעלי נכסים, אינטגרטורים, ספקי שירות ויצרני מוצרים. [מקור רשמי](#)

11. שער איכות לפרק

- המאזין מסוגל לצייר את מסלול הנתון מהשדה דרך I/O-1 Task ועד לפלט הפיזיקלי.
- הפרק מפריד בין Task Period, זמן ביצוע, Jitter, עדכון I/O וזמן תגובה כולל.
- IPC-1 PLC, PAC מושווים לפי דרישות ולא לפי סיסמאות.
- נתוני Retentive והתנעה מחדש מוצגים כהחלטות תכן מפורשות.
- IEC 61131-3:2025 מוצג במדויק בלי להבטיח ניידות מלאה בין יצרנים.
- יתירות מתוארת כארכיטקטורת מערכת ולא כ-CPU נוסף בלבד.
- הסיום מוביל ישירות לפרק 3 על חיישנים, איכות אות וכשלי מדידה.